

APPLICATION NOTE

Linux 系统 ATBM wifi6 iwpriv 命令使用说明



ATBM60XX

1x1 802.11b/g/n/ax

Wi-Fi 芯片

Table of contents

1	WIFI6 相关命令	3
1.1	设置固定 tx 速率	3
1.2	设置 edca 参数	4
1.3	获取 edca 参数	6
1.4	设置 cca 门限	6
1.5	获取 cca 门限	7
1.6	设置信号发射功率 1	7
1.7	设置信号发射功率 2	7
1.8	设置目标发射功率	8
1.9	按 b/g/n/ax 模式增加发射功率	9
1.10	按带宽方式增加发射功率	10
1.11	获取发送功率	11
1.12	通过配置文件设置不同模式 b/gn/ax 不同速率的功率配置	12
1.13	通过配置文件设置 efuse 功率因子配置功率	14
1.14	获取当前的配置文件信息	15
1.15	设置 Agc init gain	16
1.16	设置最大发包时间	17
1.17	设置 rts 门限, txop 时间和 txop 中 mpdu 个数	17
1.18	设置 A-mpdu 包中最大的 mpdu 个数	18
1.19	设置收到 ER SU 包回包模式	18
1.20	重新加载 firmware(USB wifi)	18
1.21	设置信道频点	18
1.22	获取吞吐率	19
1.23	获取信号强度	19
1.24	获取当前连接状态	20
1.25	获取 wifi mac 地址	20
1.26	设置自适应认证功能	20
1.27	发送单频	20

AN9310

Doc Rev: 3.2

Released:2023-08-14

1.28	停止发送单频.....	21
1.29	获取当前的 efuse.....	21
1.30	设置 efuse 频偏.....	21
1.31	设置 efuse 功率.....	22
1.32	设置 efuse mac 地址.....	23
1.33	Manager 模式下监听 beacon.....	23
1.34	设置 rts threshold.....	23
1.35	获取 rts threshold.....	24
1.36	获取信道空闲命令.....	24
1.37	获取接收灵敏度结果数据.....	24
1.38	白名单过滤.....	25
(1)	设置接收 beacon/probe req 帧.....	25
(2)	设置接收携带自定义 IE 的帧.....	25
(3)	清除所有的过滤条件.....	26
(4)	查询已经设置的过滤条件.....	26
1.39	获取 efuse 区域的第一个 mac 地址.....	26
1.40	打开 cfo 自动校准功能.....	26
1.41	设置固定信道扫描.....	26
1.42	获取连接上设备支持的速率.....	27
1.43	设置 power save 模式.....	28
1.44	最佳信道选择.....	28
1.45	支持 ifconfig 修改 mac 地址.....	29
1.46	获取当前的发包速率.....	30
1.47	获取出厂的 efuse.....	30
1.48	获取当前的工作信道.....	30
1.49	设置国家码.....	30
1.50	获取国家码.....	31
1.51	获取频偏和 dcxo 寄存器.....	31
1.52	与 6441 私有数据通信.....	31
(1)	设置 probe request 插入 ssid/密码.....	31
(2)	监听 probe request 帧.....	31
(3)	获取 probe request 帧携带的数据.....	32
1.53	获取信噪比.....	32
1.54	获取底噪.....	32
1.55	连续发送携带私有数据的 probe resp.....	33
1.56	停止连续发送携带私有数据的 probe resp.....	33
1.57	修改 ap 信道.....	33
1.58	Ap 模式下手动设置 TIM.....	34
1.59	获取手动设置的 TIM 数据.....	34
1.60	获取 rts duration.....	35

作者	版本	说明	时间
Yuzhihuang	3.0	WIFI6 相关命令	20230717
Yuzhihuang	3.1	增加 tim 设置相关命令	20230808

Yuzhihuang	3.2	补充一些命令	20230814

1 WIFI6 相关命令

1.1 设置固定 tx 速率

指令	<code>iwpriv wlan0 common tx_rate,idx</code>
参数	Idx: 表示为速率 id 号 设置 255 表示使用自动速率控制，设置下面的值表示固定速率 <pre> #define RATE_INDEX_B_1M 0 #define RATE_INDEX_B_2M 1 #define RATE_INDEX_B_5_5M 2 #define RATE_INDEX_B_11M 3 #define RATE_INDEX_PBCC_22M 4 // not supported/unused #define RATE_INDEX_PBCC_33M 5 // not supported/unused #define RATE_INDEX_A_6M 6 #define RATE_INDEX_A_9M 7 #define RATE_INDEX_A_12M 8 #define RATE_INDEX_A_18M 9 #define RATE_INDEX_A_24M 10 #define RATE_INDEX_A_36M 11 #define RATE_INDEX_A_48M 12 #define RATE_INDEX_A_54M 13 </pre>

	<pre> #define RATE_INDEX_N_6_5M 14 #define RATE_INDEX_N_13M 15 #define RATE_INDEX_N_19_5M 16 #define RATE_INDEX_N_26M 17 #define RATE_INDEX_N_39M 18 #define RATE_INDEX_N_52M 19 #define RATE_INDEX_N_58_5M 20 #define RATE_INDEX_N_65M 21 #define RATE_INDEX_N_MCS32_6M 22 #define RATE_INDEX_HE_MCSER0 23 #define RATE_INDEX_HE_MCSER1 24 #define RATE_INDEX_HE_MCSER2 25 #define RATE_INDEX_HE_MCS0 26 #define RATE_INDEX_HE_MCS1 27 #define RATE_INDEX_HE_MCS2 28 #define RATE_INDEX_HE_MCS3 29 #define RATE_INDEX_HE_MCS4 30 #define RATE_INDEX_HE_MCS5 31 #define RATE_INDEX_HE_MCS6 32 #define RATE_INDEX_HE_MCS7 33 #define RATE_INDEX_HE_MCS8 34 #define RATE_INDEX_HE_MCS9 35 #define RATE_INDEX_HE_MCS10 36 #define RATE_INDEX_HE_MCS11 37 </pre>
功能描述	
返回值	成功返回 0，失败返回非 0
指令示例	设置固定速率为 MCS10 iwpriv wlan0 common tx_rate,36

1.2 设置 edca 参数

命令	iwpriv wlan0 common edca_params,queue,aifs,cw_min,cw_max,txop
参数说明	<pre> queue: 0:voice traffic 1:video traffic 2:best effort 3:background trafic Sample 命令: 1, Set 命令 iwpriv wlan0 common edca_params,0,2,3,255,100 2, get 命令 </pre>

	iwpriv wlan0 common get_edca_params txop; 传输机会, 参数值为 TXOPlimit, 代表占用信道发包最长时间, 取值范围: 0~65535, 取值为 0 代表只发送一个 MPDU. 如果当前 txop 设置为 0 时, 除非是专门认证的固件, 否则 txop 会更加当前使用的速率自动计算出一个合适的值, txop 设置的值就是 rts_duration 值。 cw_min; 最小竞争窗口, 越小的 CWmin 其优先级越高, 取值范围: 2~15 cw_max; 最大竞争窗口, 越小的 CWmax 其优先级越高, CWmax > CWmin, 取值范围: 3~1023 aifs; EDCA 模式的 QSTA 要获得传输机会时, 必须等待的信道空闲时间, 时间越大代表优先级越低, 取值范围: 2~15 注意: sta 这边配置了一套 edca 参数, 如果该 sta 连上 ap, 那么 sta 这边原本配置的 EDCA 参数会被刷新为 ap 的 EDCA 参数。 连接上 ap 以后重新设置会再次生效。 抓包看变化不明显, 主要在本地会生效, 自己的发包策略会变化。
返回值	Set 命令 成功返回 0, 失败返回 -1 get 命令 <pre>[atbm_log]:[voice_traffic] [atbm_log]:aifns : 1 [atbm_log]:cwMax : 7 [atbm_log]:cwMin : 3 [atbm_log]:txOpLimit : 1504 [atbm_log]:[video_traffic] [atbm_log]:aifns : 1 [atbm_log]:cwMax : 15 [atbm_log]:cwMin : 7 [atbm_log]:txOpLimit : 3008 [atbm_log]:[best_effort] [atbm_log]:aifns : 3 [atbm_log]:cwMax : 63 [atbm_log]:cwMin : 15 [atbm_log]:txOpLimit : 0 [atbm_log]:[background_traffic] [atbm_log]:aifns : 7 [atbm_log]:cwMax : 1023 [atbm_log]:cwMin : 15 [atbm_log]:txOpLimit : 0</pre>
例子	Set 命令 <pre>iwpriv wlan0 common edca_params,0,2,3,255,100</pre> get 命令 <pre>iwpriv wlan0 common get_edca_params</pre>

1.3 获取 edca 参数

命令	<code>iwpriv wlan0 common get_edca_params</code>
参数说明	无
返回值	edca 参数中的 txop 值就是 rts duration 参数，不同的发包优先级对应的有不同的 rts_duration get 命令 <pre>[atbm_log]:[voice_traffic] [atbm_log]:aifns : 1 [atbm_log]:cwMax : 7 [atbm_log]:cwMin : 3 [atbm_log]:txOpLimit : 1504 [atbm_log]:[video_traffic] [atbm_log]:aifns : 1 [atbm_log]:cwMax : 15 [atbm_log]:cwMin : 7 [atbm_log]:txOpLimit : 3008 [atbm_log]:[best_effort] [atbm_log]:aifns : 3 [atbm_log]:cwMax : 63 [atbm_log]:cwMin : 15 [atbm_log]:txOpLimit : 0 [atbm_log]:[background_traffic] [atbm_log]:aifns : 7 [atbm_log]:cwMax : 1023 [atbm_log]:cwMin : 15 [atbm_log]:txOpLimit : 0</pre>
例子	<code>iwpriv wlan0 common get_edca_params</code>

1.4 设置 cca 门限

指令	<code>iwpriv wlan0 fwcmd cca,1,<set_cca_thr></code>
参数	<set_cca_thr>: 该参数只能设置【-85: -60】dB，超过这个范围参数设置无效，也就是用上次设置值 set_cca_thr（没有设置就用默认值-75dbm）；物理意义就是在能量检测 cca 门限值。 注意，我们芯片检测信道空闲或者繁忙有两个方法：收到一个包（包检测方法）或者检测到能量比较大（能量检测方法）都会产生信道忙，只有没收到包且能量小才会产生信道空闲。 这里设置 CCa 门限只是设置能量检测的门限，值越大（比如最大-60dBm）能量检测越

	容易出信道空闲；如果在空气中有很多包或者在跑吞吐率，则设这个 cca 门限基本没用，因为收到包就触发信道忙，根本不走能量检测逻辑。
功能描述	设置 CCA 门限。
返回值	打印: <code>ccadBm %d %d</code> : 第一个参数表示设置值，第二个参数表示实际生效值，如果第一个参数在【-85: -60】区间，第二个参数和第一个参数一样，否则不生效第二个参数就打印上次生效值。
指令示例	<code>iwpriv wlan0 fwcmd cca,1,-70</code> 设置信道忙门限为-70dbm

1.5 获取 cca 门限

指令	<code>iwpriv wlan0 common get_cca_threshold</code>
参数	
功能描述	获取 CCA 门限。
返回值	打印: <code>cca_threshold:%d</code> 失败打印: <code>cca_threshold get fail</code>
指令示例	<code>iwpriv wlan0 fwcmd common get_cca_threshold</code>

1.6 设置信号发射功率 1

命令	<code>iwpriv [interface] common set_ladder_txpower,[idx]</code>
参数说明	interface: wlan0 或者 p2p0 idx :取值 0/3/15/63, 0 表示正常发射功率模式 15 表示超高发射功率模式 63 表示最高发射功率模式 注意: 如果功率增加太多，比如超过 20dBm，芯片功耗可能超过 500mA，建议增加电源驱动能力。 该命令是 2.8 命令的集合，所以两个命令会相互影响
返回值	成功打印 0，失败打印-1
例子	<pre>ifconfig wlan0 up iwpriv wlan0 fwcmd set_txpower,15</pre>

1.7 设置信号发射功率 2

命令	<code>iwpriv [interface] common set_rate_txpower_mode,[idx]</code>
参数说明	interface: wlan0 或者 p2p0 idx :取值[-16:16] 如果设置 idx 为-16，则发射功率为-8dB。 如果设置 idx 为-6，则发射功率为-3dB。 如果设置 idx 为 0，则发射功率为+0dB。 如果设置 idx 为 6，则发射功率为+3dB。

	如果设置 idx 为 16，则发射功率为+8dB。 注意： 该命令是 2.10 命令的集合，所以两个命令会相互影响 注意： 如果功率增加太多，比如超过 20dBm，芯片功耗可能超过 500mA，建议增加电源驱动能力。
返回值	无
例子	Iwpriv wlan0 fwcmd set_rate_txpower_mode,-16

1.8 设置目标发射功率

命令	iwpriv [iface] common set_txpower, 0 ,<mode>,<rate>,<bw>,<power_target>
参数说明	iface: wlan0 或者 p2p0 mode : 发送模式, 0: 11b; 1: 11a/g; 2: 11n; 3: 11ax; rate : 根据发送模式配置对应的速率索引 11b: 0: 1M; 1: 2M; 2: 5.5M; 3: 11M 11a/g: 索引号与速率值相同, 0: 6M; 7: 54M 11n: 0: mcs0; 7: mcs7; 11ax: 0: mcs0; 11: mcs11; bw : 带宽选择, 0: 20M; 1: 40M power_target : 目标功率, 有效范围[-10,20]dBm 注意： 如果功率增加太多，比如超过 20dBm，芯片功耗可能超过 500mA，建议增加电源驱动能力。 以下速率共用寄存器，所以设置一个另外一个的发射功率也会同时改变： 1M/2M 5.5M/11M 6M/MCS0 12M/MCS1 18M/MCS2 24M/MCS3 36M/MCS4 48M/MCS5 54M/MCS6 以最后设置的为准
返回值	成功打印 0，失败打印-1
例子	ifconfig wlan0 up 1、11n mcs7 20M 设置目标功率 14dBm iwpriv wlan0 common set_txpower,0,2,7,0,14 通过 iwpriv wlan0 common get_txpower 能看到 MCS7:power:[14]dBm mode:[0]dB bw:[0]dB

	2、11ax mcs7 20M 设置目标功率 18dBm <code>iwpriv wlan0 common set_txpower,0,3,7,0,18</code> 通过 <code>iwpriv wlan0 common get_txpower</code> 能看到 MCS7:power:[18]dBm mode:[0]dB bw:[0]dB
--	--

1.9 按 b/g/n/ax 模式增加发射功率

命令	<code>iwpriv [iface] common set_txpower,1,<mode>,<power_delta></code>
参数说明	<code>iface</code> : wlan0 或者 p2p0 <code>mode</code> : 发送模式, 0: 11b; 1: 11a/g; 2: 11n; 3: 11ax; <code>power_delta</code> : 调整的功率值 (dB)。当前功率调整 <code>power_delta</code> 后仍然应该在功率有效范围内。 注意: 如果功率增加太多, 比如超过 20dBm, 芯片功耗可能超过 500mA, 建议增加电源驱动能力。 11g/11n 部分速率共用寄存器, 所以设置一个另外一个发射功率也会同时改变: 6M/MCS0 12M/MCS1 18M/MCS2 24M/MCS3 36M/MCS4 48M/MCS5 54M/MCS6 以最后设置为为准
返回值	成功打印 0, 失败打印-1
例子	<code>ifconfig wlan0 up</code> 11n MCS7 HT20 默认功率为 14dBm 11b 11M 默认功率为 18dBm 1、11b 功率提高 1dB <code>iwpriv wlan0 common set_txpower,1,0,1</code> 通过 <code>iwpriv wlan0 common get_txpower</code> 能看到 1M/2M:power:[18]dBm mode:[1]dB bw:[0]dB 5.5M/11M:power:[18]dBm mode:[1]dB bw:[0]dB 2、11n 功率提高 5dB <code>iwpriv wlan0 common set_txpower,1,2,5</code>

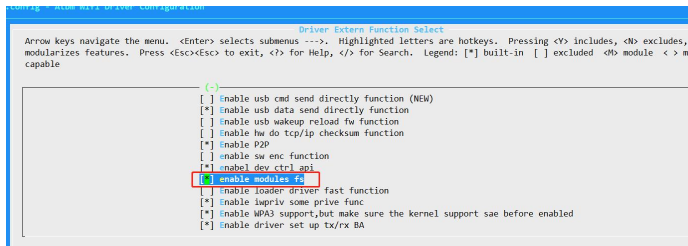
	通过 iwpriv wlan0 common get_txpower 能看到 HT20 & HT40 均有如下变化 <pre>6M/MCS0:power:[17]dBm mode:[5]dB bw:[0]dB 9M:power:[17]dBm mode:[0]dB bw:[0]dB 12M/MCS1:power:[16]dBm mode:[5]dB bw:[0]dB 18M/MCS2:power:[16]dBm mode:[5]dB bw:[0]dB 24M/MCS3:power:[15]dBm mode:[5]dB bw:[0]dB 36M/MCS4:power:[15]dBm mode:[5]dB bw:[0]dB 48M/MCS5:power:[15]dBm mode:[5]dB bw:[0]dB 54M/MCS6:power:[15]dBm mode:[5]dB bw:[0]dB MCS7:power:[14]dBm mode:[5]dB bw:[0]dB</pre> 3、11b 恢复默认功率 <pre>iwpriv wlan0 common set_txpower,1,0,0</pre> 通过 iwpriv wlan0 common get_txpower 能看到 <pre>1M/2M:power:[18]dBm mode:[0]dB bw:[0]dB 5.5M/11M:power:[18]dBm mode:[0]dB bw:[0]dB</pre> 4、11n 恢复默认功率 <pre>iwpriv wlan0 common set_txpower,1,2,0</pre> 通过 iwpriv wlan0 common get_txpower 能看到 HT20 & HT40 均有如下变化 <pre>6M/MCS0:power:[17]dBm mode:[0]dB bw:[0]dB 9M:power:[17]dBm mode:[0]dB bw:[0]dB 12M/MCS1:power:[16]dBm mode:[0]dB bw:[0]dB 18M/MCS2:power:[16]dBm mode:[0]dB bw:[0]dB 24M/MCS3:power:[15]dBm mode:[0]dB bw:[0]dB 36M/MCS4:power:[15]dBm mode:[0]dB bw:[0]dB 48M/MCS5:power:[15]dBm mode:[0]dB bw:[0]dB 54M/MCS6:power:[15]dBm mode:[0]dB bw:[0]dB MCS7:power:[14]dBm mode:[0]dB bw:[0]dB</pre>
--	---

1.10 按带宽方式增加发射功率

命令	iwpriv [iface] common set_txpower,2,<bw>,<power_delta>
参数说明	iface: wlan0 或者 p2p0 bw : 带宽选择, 0: 20M; 1: 40M power_delta : 调整的功率值 (dB)。当前功率调整 power_delta 后仍然应该在功率有效范围内。 注意:

	如果功率增加太多，比如超过 20dBm，芯片功耗可能超过 500mA，建议增加电源驱动能力。
返回值	成功打印 0，失败打印-1
例子	<pre>ifconfig wlan0 up 11n MCS7 HT20 默认功率为 14dBm 20M 带宽所有速率功率降低 2dB iwpriv wlan0 common set_txpower,2,0,-2 11n MCS7 HT20 功率为 12dBm 40M 带宽所有速率功率增加 2dB iwpriv wlan0 common set_txpower,2,1,2 11n MCS7 HT20 功率为 16dBm 20M 带宽恢复默认功率 iwpriv wlan0 common set_txpower,2,0,0 11n MCS7 HT20 功率为 14dBm</pre>

1.11 获取发送功率

命令	<pre>1、iwpriv [interface] common get_txpower 2、cat /sys/module/[wifi_module_name]/atbmfs/atbm_get_txpower</pre>
参数说明	interface: wlan0 或者 p2p0 注意： 该命令用于获取当前使用的发送功率。 Wifi_module_name: 驱动挂载的名字可通过 lsmod 查看，默认的 usb wifi 驱动名称为 ATBM606x_wifi_usb 使用该方法需要打开该配置： 
返回值	成功打印 0，失败打印-1

例子	方法 1 <pre>ifconfig wlan0 up iwpriv wlan0 common get_txpower</pre> 打印: <pre>power:[17]dBm mode:[0]dB bw:[0]dB</pre> 方法 2 <pre>cat /sys/module/ATBM606x_wifi_usb/atbmfs/atbm_get_txpower</pre> 打印: <pre>power:[17]dBm mode:[0]dB bw:[0]dB</pre> 结果说明 power 是当前 default 功率【设置目标发射功率的修改的就是这个值】， mode 是按模式调整的功率增益， bw 是按带宽调整的功率增益 芯片实际发送功率为 $current_power = (power + mode + bw)$ 注意： 当前发射功率 current_power 超过 20 请以实际仪器测试为准。
----	---

1.12 通过配置文件设置不同模式 b/gn/ax 不同速率的功率配置

命令	iwpriv wlan0 common set_rate_power,<useflag>
参数说明	useflag: 1:读取配置进行功率设置 0:复位设置 配置文件路径设置 hal_apollo/Makefile: <pre># # 可以用来选择不同模式，CONFIG_USB_AGGR_URB_TX 定义上 # 1> CONFIG_USB_AGGR_URB_TX + CONFIG_TX_NO_CONFIRM, 每个 # 2> CONFIG_USB_AGGR_URB_TX 每个urb 发送多个txframe 并且 # 3> 所有宏都不定义，每个urb 发送一个txframe 并且需要cc # 4> 定义CONFIG_USE_DMA_ADDR_BUFFER, 每个urb 发送一个tx # 5> 定义CONFIG_USE_DMA_ADDR_BUFFER+ CONFIG_TX_NO_CONFI # 6> 定义CONFIG_TX_NO_CONFIRM, 每个urb 发送一个txframe # ##对于调度慢+cpu强的板子使用 模式1 或者 模式2 ##对于调度快+cpu慢的板子使用 模式3 或者 模式6 ## 默认使用模式 3 ##### #ccflags-y += -DCONFIG_ATBM_APOLLO_5GHZ_SUPPORT #ccflags-y += -DCONFIG_ATBM_APOLLO_WAPI_SUPPORT #ccflags-y += -I\$(shell pwd)/../include/ \ -include \$(shell pwd)/../include/linux/compat-2 -DCOMPAT_STATIC ##指定配置文件路径，当配置文件存在时使用配置文件中的dcx ATBM_CONFIG_FILE="/atbm txpwer dcxo cfg.txt" ##指定配置文件路径，当配置文件存在时使用配置文件中的det ATBM_RATE_POWER_CONFIG_FILE="/set_rate_power.txt"</pre> 注意：

	如果功率增加太多，比如超过 20dBm，芯片功耗可能超过 500mA，建议增加电源驱动能力。
返回值	成功返回 0，失败返回 -1
例子	配置文件格式： <pre>target_or_delta=0+ b_1M_2M=0+ b_5_5M_11M=0+ g_6M_n_6_5M=0+ g_9M=0+ g_12M_n_13M=0+ g_18M_n_19_5M=0+ g_24M_n_26M=0+ g_36M_n_39M=0+ g_48M_n_52M=0+ g_54M_n_58_5M=0+ n_65M=0+ HE-SU_MCS0=0+ HE-SU_MCS1=0+ HE-SU_MCS2=0+ HE-SU_MCS3=0+ HE-SU_MCS4=0+ HE-SU_MCS5=0+ HE-SU_MCS6=0+ HE-SU_MCS7=0+ HE-SU_MCS8=0+ HE-SU_MCS9=0+ HE-SU_MCS10=0+ HE-SU_MCS11=0+ g_6M_n_6_5M_40M=0+ g_9M_40M=0+ g_12M_n_13M_40M=0+ g_18M_n_19_5M_40M=0+ g_24M_n_26M_40M=0+ g_36M_n_39M_40M=0+ g_48M_n_52M_40M=0+ g_54M_n_58_5M_40M=0+ n_65M_40M=0+ HE-SU_MCS0_40M=0+ HE-SU_MCS1_40M=0+ HE-SU_MCS2_40M=0+ HE-SU_MCS3_40M=0+ HE-SU_MCS4_40M=0+ HE-SU_MCS5_40M=0+ HE-SU_MCS6_40M=0+ HE-SU_MCS7_40M=0+ HE-SU_MCS8_40M=0+ HE-SU_MCS9_40M=0+ HE-SU_MCS10_40M=0+</pre> target_or_delta=0 时： 数据配置后仍应该使得发送功率在 20dbm 以内，配置的数据是 10*dB。 如果设置 b_1M_2M=10，则 1/2M 速率发送功率增加 1dB。 如果设置 g_6M_n_6_5M =-10，则 6M/6.5Ms1v 发送功率降低 1dB。 target_or_delta=1 时： 数据配置后仍应该使得发送功率在 20dbm 以内，配置的数据是 10*dB。 如果设置 b_1M_2M=180，则 1/2M 速率发射功率为 18dBm。 如果设置 g_6M_n_6_5M =140，则 6M/6.5M 速率发射功率为 14dBm 注意， 1) 该配置命令只在有烧过 efuse 模组或 EVB 才起作用。 2) 驱动在加载的时候不会默认去读取这个文件并根据配置文件内容生效。 运行过程中修改了配置文件内容，并想立即生效直接执行命令： <pre>iwpriv wlan0 common set_rate_power,1</pre> 驱动会去刷新配置并立即生效。

使用命令将设置好的配置复位:

iwpriv wlan0 common set_rate_power,0

1.13 通过配置文件设置 efuse 功率因子配置功率

命令

iwpriv wlan0 common settxpower_byfile

参数说明

```

1 #CONFIG_TX_NO_CONFIRM
2 #CONFIG_USB_AGGUR_URB_TX
3 #CONFIG_TX_NO_CONFIRM
4 #
5 # 可以用来选择不同是模式, CONFIG_USB_AGGUR_URB_TX 定义上时一定要定义CONF
6 # 1> CONFIG_USB_AGGUR_URB_TX + CONFIG_TX_NO_CONFIRM, 每个urb 发送多个txframe
7 # 2> CONFIG_USB_AGGUR_URB_TX 每个urb 发送多个txframe 并且需要confirm, 需要
8 # 3> 所有宏都不定义, 每个urb 发送一个txframe 并且需要confirm, 并且urb直接
9 # 4> 定义CONFIG_USE_DMA_ADDR_BUFFER, 每个urb 发送一个txframe 并且需要confi
10 # 5> 定义CONFIG_USE_DMA_ADDR_BUFFER+ CONFIG_TX_NO_CONFIRM, 每个urb 发送一
11 # 6> 定义CONFIG_TX_NO_CONFIRM, 每个urb 发送一个txframe 并且不需要confirm,
12 #
13 ##对于调度慢+cpu强的板子使用 模式1 或者 模式2
14 ##对于调度快+cpu慢的板子使用 模式3 或者 模式6
15 ## 默认使用模式 3
16 #####
17 #ccflags-y += -DCONFIG_ATBM_APOLLO_5GHZ_SUPPORT
18 #ccflags-y += -DCONFIG_ATBM_APOLLO_WAPI_SUPPORT
19 #ccflags-y += -I$(shell pwd)/../include/ \
20 -include $(shell pwd)/../include/linux/compat-2.6.h \
21 -DCOMPAT_STATIC
22 ##指定配置文件路径, 当配置文件存在时使用配置文件中的dcxo值不使用efuse中的
23 #ATBM_CONFIG_FILE="/atbm txpwer dcxo cfg.txt"
24 ifneq ($(IFNAME),)
25 #ccflags-y += -DCONFIG_IFNAME="\$(IFNAME)"
26 endif

```

注意:

如果功率增加太多, 比如超过 20dBm, 芯片功耗可能超过 500mA, 建议增加电源驱动能力。

返回值

成功返回 0, 失败返回 -1

例子

配置方式只有一种覆盖式, 所以在设置前需要先确认下当前在使用的 deltagain 值, 在进行相应的去调整配置文件格式:

delta_gain1:1 delta_gain2:2 delta_gain3:3 dcxo:59

计算公式:

芯片使用的 deltagain 值,CHIP_VAL ^①	配置文件配置的 deltagain 值,CONF_VAL ^②	功率增益计算公式 ^③
>16 ^④	>16 ^⑤	(CONF_VAL - CHIP_VAL)/4 ^⑥
	<16 ^⑤	(32 - CHIP_VAL)/4 + CONF_VAL / 4 ^⑥
	=16 或者 =0 ^⑤	(32 - CHIP_VAL) / 4 ^⑥
<16 ^④	>16 ^⑤	(CONF_VAL-32)/4 + (0 - CHIP_VAL)/4 ^⑥
	<16 ^⑤	(CONF_VAL - CHIP_VAL) / 4 ^⑥
	=16 或者 =0 ^⑤	(0 - CHIP_VAL) / 4 ^⑥
=16 或者 = 0 ^④	>16 ^⑤	(CONF_VAL - 32) / 4 ^⑥
	<16 ^⑤	CONF_VAL / 4 ^⑥
	=16 或者 =0 ^⑤	0 ^⑥

举例:

当前芯片使用的 deltagain 值 CHIP_VAL:

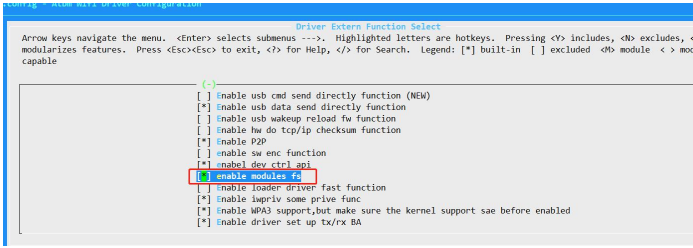
delta_gain1:27 delta_gain2:27 delta_gain3:27 dcxo:50

配置文件数据 CONF_VAL:

delta_gain1:3 delta_gain2:17 delta_gain3:16 dcxo:60

	deltagain1 增益变化: CHIP_VAL > 16 CONF_VAL < 16 Deltagain1 增益: $(32 - \text{CHIP_VAL})/4 + \text{CONF_VAL} / 4$ $= (32-27)/4 + 3/4 = 2\text{dB}$ Deltagain2 增益变化: CHIP_VAL > 16 CONF_VAL > 16 Deltagain2 增益: $(\text{CONF_VAL} - \text{CHIP_VAL})/4$ $= (17-27)/4 = -2.5\text{dB}$ Deltagain3 增益变化: CHIP_VAL > 16 CONF_VAL = 16 Deltagain3 增益: $(32 - \text{CHIP_VAL}) / 4$ $= (32 - 27)/4 = 1.25\text{dB}$ 数据都不会写到 efuse ,该配置掉电就没了, 不会保存到 ROM PS : 驱动在加载的时候会默认去读取这个文件并根据配置文件内容生效。 运行过程中修改了配置文件内容, 并想立即生效直接执行命令: <code>iwpriv wlan0 common settxpower_byfile</code>
--	--

1.14 获取当前的配置文件信息

命令	1、<code>iwpriv [interface] common get_cfg_txpower</code> 2、<code>cat /sys/module/[wifi_module_name]/atbmfs/atbm_get_cfg_power</code>
参数说明	interface: wlan0 或者 p2p0 注意: 该命令用于获取当前使用的发送功率。 Wifi_module_name: 驱动挂载的名字可通过 <code>lsmod</code> 查看, 默认的 usb wifi 驱动名称为 <code>ATBM606x_wifi_usb</code> 使用该方法需要打开该配置: 
返回值	成功打印 0, 失败打印-1

例子	方法 1 <pre>ifconfig wlan0 up iwpriv wlan0 common get_txpower</pre> 方法 2 <pre>cat /sys/module/ATBM606x_wifi_usb/atbmfs/atbm_get_cfg_power</pre> 两个方法打印一样： <pre>[atbm:rog]:ne-50_mcs11_40m=0 wlan0 common:b_1M_2M=0 b_5_5M_11M=0 g_6M_n_6_5M=0 g_9M=0 g_12M_n_13M=0 g_18M_n_19_5M=0 g_24M_n_26M=0 g_36M_n_39M=0 g_48M_n_52M=0 g_54M_n_58_5M=0 n_65M=0 HE-SU_MCS0=0 HE-SU_MCS1=0 HE-SU_MCS2=0 HE-SU_MCS3=0 HE-SU_MCS4=0 HE-SU_MCS5=0 HE-SU_MCS6=0 HE-SU_MCS7=0 HE-SU_MCS8=0 HE-SU_MCS9=0 HE-SU_MCS10=0 HE-SU_MCS11=0 g_6M_n_6_5M_40M=0 g_9M_40M=0 g_12M_n_13M_40M=0 g_18M_n_19_5M_40M=0 g_24M_n_26M_40M=0 g_36M_n_39M_40M=0 g_48M_n_52M_40M=0 g_54M_n_58_5M_40M=0 n_65M_40M=0 HE-SU_MCS0_40M=0 HE-SU_MCS1_40M=0 HE-SU_MCS2_40M=0 HE-SU_MCS3_40M=0 HE-SU_MCS4_40M=0 HE-SU_MCS5_40M=0 HE-SU_MCS6_40M=0 HE-SU_MCS7_40M=0 HE-SU_MCS8_40M=0 HE-SU_MCS9_40M=0 HE-SU_MCS10_40M=0 HE-SU_MCS11_40M=0 delta_gain1:27 delta_gain2:27 delta_gain3:27 dcxo:50</pre>
----	---

1.15 设置 Agc init gain

指令	iwpriv wlan0 fwcmd agc,value
参数	value: 0 表示不设置 agc init gain, agc init gain 完全按照固件逻辑跑; value: 1 表示 agc init gain 固定 60dB, 看底噪必须把该参数设置为 60. Value: 【2: 2: 60】, agc init gain 只能是偶数, 如果值在这个范围内, agc gain 固定为该值, 一般 debug 用。
功能描述	Debug 用, 一般离 ap 越近该值越小越好, 最小不要小于 24dB。
返回值	可以输入命令 iwpriv wlan0 fwcmd ccastart,1 或 3, 打印 inig 就可以看到命令生效效果
指令示例	iwpriv wlan0 fwcmd agc,0: agc init gain 就是正常工作状态调整 iwpriv wlan0 fwcmd agc,1: agc init gain 固定 60dB iwpriv wlan0 fwcmd agc,24, agc init gain 固定 24dB

1.16 设置最大发包时间

指令	<code>iwpriv wlan0 fwcmd txtime,params</code>
参数	Params: 单位是 ms; 值只能在 100~10*1024 这个范围。否则不生效
功能描述	设置包最大发送时间, 默认是 2000ms
返回值	需要打开 <code>iwpriv wlan0 fwdbg 1</code> , 后在进行设置会看到打印 <code>LiveTime%d,%d</code> : 第一个参数打印输入参数, 第二个参数打印表示实际生效值
指令示例	<code>iwpriv wlan0 fwcmd txtime,128</code> : 最大发包时间设置 128ms <code>iwpriv wlan0 fwcmd txtime,5000</code> : 最大发包时间设置 5000ms

1.17 设置 rts 门限, txop 时间和 txop 中 mpdu 个数

指令	<code>iwpriv wlan0 fwcmd rts,get_or_set,rts_time_thr,rts_duration,mpdu_num,rts_cf_end</code>
参数	<code>get_or_set</code>: 0 打印该命令设置情况, 具体见下面返回值; 1 设置该命令。 <code>rts_time_thr</code>: 只有参数 <code>get_or_set</code> 为 1 才生效, 设置发 rts 包的时间门限, 单位是 us。该参数设置不能超过 10000 (对应 10ms), 如果超过就限制成 10000; 该参数设置为 0 的时候, rts 门限就是和 ap 协商的。IPC 固件该参数默认是 500, 也就是只要组包长度超过 500us 就会发 rts 包; 电视固件该参数是和 AP 协商出来的。 <code>rts_duration</code>: 只有参数 <code>get_or_set</code> 为 1 才生效, 设置 rts 保护间隔, 也叫做 txop 时间, 单位是 us。该参数不能超过 8191, 如果超过就限制成 8191。如果该参数是 0, 用的是 ap 协商的时间; 如果该参数小于 500, 用默认的 txop 时间 (默认是动态变化的, 最小 2000us, 最大 8000us); 该参数大等于 500, 才生效用于 txop 时间。 <code>mpdu_num</code>: 只有参数 <code>get_or_set</code> 为 1 才生效, 表示设置 txop 时间内最多元勋发送 mpdu 包个数, 最大不能超过 63 个。如果该参数是 0, 用的是默认值 (63)。 <code>rts_cf_end</code>: 是否发 cf end, IPC 固件默认是 1, 电视固件默认是 0。
功能描述	设置 rts 门限, txop 时间和 txop 中 mpdu 个数
返回值	<code>RTS:Thr%dus,Txop%dus,MPDU%d,CFEnd%d</code>
指令示例	<code>iwpriv wlan0 fwcmd rts,0</code>: 获取 rts 命令设置情况, 会打印上面返回值 <code>iwpriv wlan0 fwcmd rts,1,500,1,0,1</code> 设置发 rts 包时间是 500us, rts duration 和 mpdu 个数用默认参数, 发送 CF END 包。 <code>iwpriv wlan0 fwcmd rts,1,1000,3000,32,0</code> 设置发 rts 包时间是 1000us, rts duration 设置为 3000us, mpdu 个数设置为 32, 不发送 CF END 包。

1.18 设置 A-mpdu 包中最大的 mpdu 个数

指令	<code>iwpriv wlan0 fwcmd ampdu,num</code>
参数	num: 只能取 1~64, 超过该值用默认的 20
功能描述	设置一次组 ampdu 最多 mpdu 个数, 注意这个参数是针对包长度是 1538byte 的个数, 如果是跑 tx, 发送包长度一般是 1538byte, 那么每次聚合 ampdu 最多 mpdu 个数是 num; 但如果跑 tcp rx, 一般 tcp ack 包长度是 90byte, 那么实际组 ampdu 最多 mpdu 个数 $1538 * num / 90$
返回值	AMPDUMaxNum: %d
指令示例	<code>iwpriv wlan0 fwcmd ampdu,8</code> : 每次组 ampdu 最大 mpdu 个数是 8 个

1.19 设置收到 ER SU 包回包模式

指令	<code>iwpriv wlan0 fwcmd ersu,mode</code>
参数	mode: 只能取 0 或 1, 默认是 0
功能描述	设置收到 ER SU 包回的响应包 (CTS/ACK/BA) 是 legacy 6M 包还是 ER SU 包
返回值	ERSU %d
指令示例	<code>iwpriv wlan0 fwcmd ersu,0</code> : 收到 ER SU 包回响应包是 legacy 包 <code>iwpriv wlan0 fwcmd ersu,1</code> : 收到 ER SU 包回响应包是 ER SU 包

1.20 重新加载 firmware(USB wifi)

指令	<code>iwpriv wlan0 common fw_reset,wifi_ble_comb</code>
参数	Wifi_ble_comb: 1: 重新加载 wifi ble 共存固件 0: 重新加载独立的 wifi 固件
功能描述	重新加载 firmware
返回值	返回 0
指令示例	<code>iwpriv wlan0 common fw_reset,1</code>

1.21 设置信道频点

命令	1、 <code>iwpriv [interface] fwcmd set_freq,[chan_number],[chan_value]</code> 2、 <code>iwpriv [interface] set_freq [chan_number],[chan_value]</code>
参数说明	Interface : wlan0 或者 p2p0 Chan_number: 信道号, 取值 [1~13] Chan_value : 频点参数, 取值 2380 或者 2504, 或者正常的 1~13 频点

	注意： 使用命令 2 设置特殊频点可以在文件系统查询到。 <pre>cat /sys/modules/atbmxxx/paramaters/wifi_freq</pre>
返回值	
例子	1、设置特殊频点 <pre>ifconfig wlan0 up iwpriv wlan0 fwcmd set_freq,1,2380</pre> 2、设置回正常频点 <pre>ifconfig wlan0 up iwpriv wlan0 fwcmd set_freq,1,2412</pre>

注：

(1) 不同的信道不能设置相同的频点，驱动也需要做这方面的判断，预防出错。

(2) 正常频点：

2412/2417/2422/2427/2432/2437/2442/2447/2452/2457/2462/2467/2472/2484

(3) 特殊频点：2380/2504

1.22 获取吞吐率

命令	<code>iwpriv [interface] get_tp_rate [sta mac addr]</code>
参数说明	Interface : wlan0 或者 p2p0 sta mac addr: 设备本身做为 ap 模式需要获取连接的 sta 的当前环境的吞吐量，该参数对应的是 sta 的 mac 地址，sta 模式该参数可以为空
返回值	单位是 bits/s
例子	<pre>ifconfig wlan0 up 连接上需要 ping、或者传输数据才行 iwpriv wlan0 get_tp_rate</pre>

1.23 获取信号强度

命令	<code>iwpriv [interface] common get_rssi [sta mac addr]</code>
参数说明	Interface : wlan0 或者 p2p0 sta mac addr: 设备本身做为 ap 模式需要获取连接的 sta 的信号强度，该参数对应的是 sta 的 mac 地址，sta 模式该参数可以为空
返回值	当前的 rssi 值
例子	<pre>ifconfig wlan0 up iwpriv wlan0 common get_rssi</pre>

1.24 获取当前连接状态

命令	<code>iwpriv [interface] get_state</code>
参数说明	Interface: wlan0 或者 p2p0
返回值	打印出来 0 或者 1, 0 未连接状态 1 已连接状态
例子	<code>ifconfig wlan0 up</code> <code>iwpriv wlan0 get_state</code>

1.25 获取 wifi mac 地址

命令	<code>iwpriv [interface] getmac</code>
参数说明	interface: wlan0 或者 p2p0
返回值	打印出来 wlan0 或者 p2p0 的 mac 地址: dc:29:19:00:25:7f
例子	<code>Iwpriv wlan0 getmac</code>

1.26 设置自适应认证功能

命令	<code>iwpriv wlan0 fwcmd set_adaptive,value</code>
参数说明	Value = 1 开启自适应功能 Value = 0 关闭自适应功能 需要注意协议规定干扰信号在-70dBm , ATBM WIFI 限制的门限为-75dBm , 干扰小于-75dBm 测试就会有问题的
返回值	Lmac 有打印: adatpive value
例子	

1.27 发送单频

命令	<code>iwpriv wlan0 common singletone,<channel></code>
参数说明	Channel: 发送 singletone 时的中心频点, 取值范围 1~14 Singletone 命令与 etf start_tx 以及 start_rx 命令互斥, 发送 singletone 前必须先停止 start_tx 或者 start_rx
返回值	无
例子	发送没调制的 1 信道信号: <code>iwpriv wlan0 common singletone,1</code>

1.28 停止发送单频

命令	<code>iwpriv wlan0 stop_tx</code>
参数说明	无
返回值	无
例子	

1.29 获取当前的 efuse

命令	<code>iwpriv wlan0 common getEfuse</code>
参数说明	无
返回值	Get efuse data is [1,80,5,4,4,6,0,0,dc:29:19:4c:81:e7] 80: 是 dcxo 频偏 5,4,4 : gain1,gain2,gain3 dc:29:19:4c:81:e7:mac 地址
例子	需要注意在所有设置 efuse 之前都必须先读取出来数据。 所以写 efuse 的流程按照如下规则： 【一定要按照下面的步骤，不然会设置失败】 1、get efuse 2、set efuse

1.30 设置 efuse 频偏

命令	<code>iwpriv wlan0 common setEfuse_dcxo,<dcxo>,write_rom</code>
参数说明	Dcxo : 设置频偏因子的值 , 范围 0~128 write_rom: 是否断电保存数据。1 保存, 0 不保存。 通过 get 回来的数据为基准数据: 该值改大, 频偏往小了偏 该值改小, 频偏往大了偏
返回值	无
例子	<code>iwpriv wlan0 common getEfuse</code> 返回值: Get efuse data is [1,80,5,4,4,6,0,0,dc:29:19:4c:81:e7] 当前频偏为: -3 <code>iwpriv wlan0 common setEfuse_dcxo,70,0</code> 当前频偏为: 12 (规律和晶体的曲线相关, 整体呈负相关)

1.31 设置 efuse 功率

命令	iwpriv wlan0 common setEfuse_deltagain, <delta_gain1>,<delta_gain2>,<delta_gain3>,write_rom
参数说明	delta_gain1: 低信道功率因子 : 0~31 delta_gain2: 中信道功率因子 : 0~31 delta_gain3: 高信道功率因子 : 0~31 write_rom: 是否断电保存数据。1 保存, 0 不保存 取值 0~15 功率增大 取值 16~31 功率减小 注意: 如果功率增加太多, 比如超过 20dBm, 芯片功耗可能超过 500mA, 建议增加电源驱动能力。
返回值	无
例子	1、芯片内部的 efsue 小于 16 iwpriv wlan0 common getEfuse 返回值: Get efuse data is [1,80,5,4,4,6,0,0,dc:29:19:4c:81:e7] iwpriv wlan0 common setEfuse_deltagain,9,8,8,1 功率因子修改前后的功率差别: 低信道功率增加: $(9-5)/4 = 1\text{dB}$ 中信道功率增加: $(8-4)/4 = 1\text{dB}$ 高信道功率增加: $(8-4)/4 = 1\text{dB}$ iwpriv wlan0 common setEfuse_deltagain,17,18,19,1 功率因子修改后的功率差别: 低信道功率减少: $(17-32)/4 - 5/4 = -5\text{dB}$ 中信道功率减少: $(18-32)/4 - 4/4 = -4.5\text{dB}$ 高信道功率减少: $(19-32)/4 - 4/4 = -4.25\text{dB}$ 2、芯片内部的 efsue 大于 16 iwpriv wlan0 common getEfuse 返回值: Get efuse data is [1,80,25,24,24,6,0,0,dc:29:19:4c:81:e7] iwpriv wlan0 common setEfuse_deltagain,9,8,8,1 功率因子修改前后的功率差别: 低信道功率增加: $9/4 - (25 - 32)/4$ 中信道功率增加: $8/4 - (24 - 32)/4$ 高信道功率增加: $8/4 - (24 - 32)/4$

	<pre>iwpriv wlan0 common setEfuse_deltagain,17,18,19,1</pre> 功率因子修改后的功率差别： 低信道功率减少： $(17-32)/4 - (25 - 32)/4$ 中信道功率减少： $(18-32)/4 - (24 - 32)/4$ 高信道功率减少： $(19-32)/4 - (24 - 32)/4$
--	--

1.32 设置 efuse mac 地址

命令	<code>iwpriv wlan0 common setEfuse_mac,<mac 地址></code>
参数说明	Mac 地址：格式必须是 xx:xx:xx:xx:xx:xx
返回值	无
例子	<pre>iwpriv wlan0 common getEfuse</pre> 返回值：Get efuse data is [1,80,5,4,4,6,0,0,dc:29:19:4c:81:e7] <pre>iwpriv wlan0 common setEfuse_mac,10:02:03:04:05:01</pre> 修改完成没有立即生效，需要立即生效需要重新卸载/加载下驱动。

1.33 Manager 模式下监听 beacon

命令	<code>iwpriv wlan0 sta_channel <channel></code>
参数说明	Channel : 信道,取值范围[1,14] 注意： 打开宏 CONFIG_ATBM_STA_LISTEN
返回值	无
例子	监听 1 信道的 beacon <pre>ifconfig wlan0 up</pre> <pre>iwpriv wlan0 sta_channel 1</pre>

1.34 设置 rts threshold

命令	<code>iwpriv wlan0 rts_threshold package_len</code>
参数说明	package_len:设置触发 rts 的包长度，取值范围(0,2000) 设置超过 2000 关发闭 rts
返回值	无
例子	关闭发 rts: <pre>iwpriv wlan0 rts_threshold 2000</pre> 包长超过 1000 触发 rts

	<code>iwpriv wlan0 rts_threshold 1000</code>
--	--

1.35 获取 rts threshold

命令	<code>iwpriv wlan0 common get_rts_threshold</code>
参数说明	无
返回值	无
例子	<code>iwpriv wlan0 common get_rts_threshold</code>

1.36 获取信道空闲命令

命令	<code>iwpriv wlan0 common get_channel_idle</code>
参数说明	
返回值	current_idle:value，其中信道空闲 value 取值范围是 1~1024。 值越大，信道越空闲，信道空闲测量底层是每 2~3s 才计算一次。 如果信道空闲值小于 100，建议换信道或者降低码率。 当打印信道空闲值为 0 时，表示信道测量还没测出来，可以 3s 后再测量一次。 当读取信道空闲值和上次值一样，表示当前的信道空闲还没计算出来，这个时候打印的还是上次测量的信道空闲值 由于一次信道空闲值可能不准，建议读取多次信道空闲值做平均，注意做平均时需要去掉和上次信道空闲值一样的值。 比如： 多次读取信道空闲值时： current_idle:100, current_idle:200, current_idle:200, current_idle:150, current_idle:200, current_idle:100, 平均出来值应该是 $(100+200+150+200+100)/5=150$， 而不是 $(100+200+200+150+200+100)/6=950/6$
例子	

1.37 获取接收灵敏度结果数据

命令	<code>iwpriv wlan0 rx_statis</code>
参数说明	该命令必须要在已经执行了,接收灵敏度统计命令 <code>iwpriv wlan0 start_rx chan,is_40M</code> 前提下执行的。

	直接获取当前时刻统计的数据。 如果没有执行接收灵敏度统计命令，直接执行 <code>iwpriv wlan0 rx_stat</code> 获取结果会执行失败。
返回值	成功返回 0，失败返回 -1
例子	

1.38 白名单过滤

(1) 设置接收 beacon/probe req 帧

命令	<code>iwpriv wlan0 common filter_frame,<idx></code>
参数说明	Idx : 80 接收 beacon 帧 40 接收 probe req 帧 需要启动 hostapd 或者 wpa_supplicant
返回值	成功返回 0，失败返回 -1
例子	1、设置过滤 beacon 帧 <code>Iwpriv wlan0 common filter_frame,80</code> 2、设置过滤 probe req 帧 <code>Iwpriv wlan0 common filter_frame,40</code> 数据处理函数 <code>hal_apollo/mac80211/special_filter.c</code> <code>ieee80211_special_filter_rx_package_handle</code>

(2) 设置接收携带自定义 IE 的帧

命令	<code>iwpriv wlan0 common filter_ie,<id>,<oui1>,<oui2>,<oui3></code>
参数说明	Id:IE 对应的 id 号,十进制 Oui1-oui2-oui3:IE 对应的 oui, 十进制 Oui1,oui2,oui3 可为空 需要启动 hostapd 或者 wpa_supplicant
返回值	成功返回 0，失败返回 -1
例子	1、设置过滤 221 字段的帧 <code>Iwpriv wlan0 common filter_ie,221</code> 2、设置过滤 221 字段 oui 为 5-6-7-的帧 <code>Iwpriv wlan0 common filter_ie,221,5,6,7</code> 数据处理函数 <code>hal_apollo/mac80211/special_filter.c</code> <code>ieee80211_special_filter_rx_package_handle</code>

(3) 清除所有的过滤条件

命令	<code>iwpriv wlan0 common filter_clear</code>
参数说明	无
返回值	成功返回 0，失败返回 -1
例子	需要注意如果设置使用的网口是 p2p0 那么清除也需要使用 p2p0 网口进行清除数据

(4) 查询已经设置的过滤条件

命令	<code>iwpriv wlan0 common filter_show</code>
参数说明	无
返回值	成功返回 0，失败返回 -1
例子	

1.39 获取 efuse 区域的第一个 mac 地址

命令	<code>iwpriv wlan0 common get_first_mac</code>
参数说明	无
返回值	成功返回 0，失败返回 -1
例子	

1.40 打开 cfo 自动校准功能

命令	<code>iwpriv wlan0 fwcmd cfo,p1</code>
参数说明	value: 1 打开校准功能; value: 0 关闭校准功能。
返回值	成功返回 0，失败返回 -1
例子	打开自动频偏校准: <code>Iwpriv wlan0 fwcmd cfo,1</code> 关闭自动频偏校准: <code>Iwpriv wlan0 fwcmd cfo,0</code>

1.41 设置固定信道扫描

命令	<code>iwpriv wlan0 common ap_conf,channel</code>
参数说明	驱动需要打开宏 CONFIG_ATBM_SUPPORT_AP_CONFIG channel:1~14
返回值	成功返回 0，失败返回 -1
例子	设置固定 1 信道扫描 <code>iwpriv wlan0 common ap_conf,3</code> <code>iwlist wlan0 scan</code> 此时只会扫描 3 信道

	关闭固定信道扫描 <code>iwpriv wlan0 common ap_conf,0</code> <code>iwlist wlan0 scan</code> 此时会扫描 1~14 信道
--	---

1.42 获取连接上设备支持的速率

命令	<code>iwpriv wlan0 common stations_show</code>
参数说明	无
返回值	成功返回 0，失败返回 -1
例子	<pre> ~ # iwpriv wlan0 common stations_show wlan0 common:station show --> mac[fc:be:7b:0c:8f:65],rssi[-77],bg[fff],11n[ff] ~ # iwpriv wlan0 common stations_show wlan0 common:station show --> mac[fc:be:7b:0c:8f:65],rssi[-76],bg[fff],11n[ff] ~ # iwpriv wlan0 common stations_show wlan0 common:station show --> mac[fc:be:7b:0c:8f:65],rssi[-76],bg[fff],11n[ff] ~ # iwpriv wlan0 common stations_show wlan0 common:station show --> mac[fc:be:7b:0c:8f:65],rssi[-76],bg[fff],11n[ff] ~ # iwpriv wlan0 common stations_show wlan0 common:station show --> mac[fc:be:7b:0c:8f:65],rssi[-76],bg[fff],11n[ff] ~ # iwpriv wlan0 common stations_show wlan0 common:station show --> mac[fc:be:7b:0c:8f:65],rssi[-75],bg[fff],11n[ff] </pre> 说明： mac: 连接设备的 mac 地址 rssi: 信号强度 bg/11n 协商的速率： 其中：bg[fff] --> 转成二进制共 12 位对应 b/g 12 个速率： 从低到高对应速率：[1M 2M 5.5M 11M 6M 9M 12M 18M 24M 36M 48M 54M] 如上图：bg[fff] 代表支持 b/g 全速率 如果：bg[55f] 代表支持 [1M 2M 5.5M 11M 6M 12M 24M 48M] 其中：11n[ff] 从低到高对应的是 MCS0~MCS7 如上图：11n[ff] 代表当前连接支持 MCS0~MCS7 如果：11n[53] 0101 0011 代表支持 [MCS0 MCS1 MCS4 MCS6] <pre> /tmp/wifi6 # /tmp/wifi6 # iwpriv wlan0 common stations_show [atbm_log]:wsm_generic_req_confirm:status err[4] wlan0 common:station show --> mac[10:0c:6b:1f:4e:54],rssi[-13],bg[fff],11n[ff] ax[fff] /tmp/wifi6 # </pre> 如果连接的路由器支持 WIFI6 会增加 ax[%x] 打印： ax[fff] => 支持 MCS0~MCS11 如果是 ax[1ff] => 0001 1111 1111 => 支持 MCS0~MCS8

1.43 设置 power save 模式

命令	<code>iwpriv wlan0 common power_save,value</code>
参数说明	Value: 1:开启休眠功能 0:关闭休眠功能 进入休眠以后会发 null data 包, 包里面 power save 位为 1
返回值	成功返回 0, 失败返回-1
例子	

1.44 最佳信道选择

命令	<code>iwpriv wlan0 best_ch_scan < BestChannelSelect_t ></code>
参数说明	有特殊信道使用需要打开宏: CONFIG_ATBM_5G_PRETEND_2G 参数说明: <pre> #define CHANNEL_NUM 18 //信道个数 typedef struct _best_ch_scan_result_ { //每个信道 AP 的个数 unsigned int channel_ap_num[CHANNEL_NUM]; //每个信道的信道忙比例, 范围 0~128 (0: 信道全空闲, 128: 信道全忙) unsigned int busy_ratio[CHANNEL_NUM]; //每个信道比重 unsigned int weight[CHANNEL_NUM]; //统计后, 建议的信道 unsigned char suggest_ch; }Best_Channel_Scan_Result; typedef struct BestChannelSelect{ unsigned char start_channel; unsigned char end_channel; unsigned char SpecialFlag; Best_Channel_Scan_Result scan_result; }BestChannelSelect_t; </pre> 说明: start_channel; 起始统计信道 end_channel; 终止统计信道 SpecialFlag; 是否统计特殊信道 scan_result: 保存返回的结果
返回值	成功返回 0, 失败返回-1
例子	建议使用自带的 <code>iwprv_demo</code> 进行测试:

	1、iwpriv wlan0 best_ch_scan 默认统计 1~14 信道的最佳信道，但是应用层获取不到数据，内核只打印最佳信道 2、iwpriv_demo wlan0 best_ch_scan start_ch end_ch specialFlag 2.1) iwpriv_demo wlan0 best_ch_scan 1 11 0 统计 1~11 信道 2.2) iwpriv_demo wlan0 best_ch_scan 1 14 0 统计 1~14 信道 2.3) iwpriv_demo wlan0 best_ch_scan 1 14 1 统计 1~14 信道 2.4) iwpriv_demo wlan0 best_ch_scan 1 11 1 统计 1~11 信道 2.5) iwpriv_demo wlan0 best_ch_scan 1 42 0 统计 1~14 信道 2.6) iwpriv_demo wlan0 best_ch_scan 1 42 1 统计 1~14 信道 + 统计 36/38/40/42 信道 2.7) iwpriv_demo wlan0 best_ch_scan 1 36 1 统计 1~14 信道+统计 36 信道 2.8) iwpriv_demo wlan0 best_ch_scan 1 36 0 统计 1~14 信道
--	---

1.45 支持 ifconfig 修改 mac 地址

命令	ifconfig wlan0 hw ether [mac_addr]
参数说明	1.wlan0/1 网口状态必须处于 down 状态 2.只能修改 wlan0 网口的 mac 地址。 3.打开宏 <code>ccflags-y += -DCONFIG_ATBM_SUPPORT_REALTIME_CHANGE_MAC</code>
返回值	成功返回 0，失败返回-1
例子	ifconfig wlan0 hw ether 00:11:22:33:44:55

1.46 获取当前的发包速率

命令	<code>iwpriv wlan0 common get_maxrate[,mac_addr]</code>
参数说明	mac_addr: wlan0 做为 AP 模式时候使用的参数, mac_addr 为连接上 AP 的 sta mac 地址
返回值	成功返回 0, 失败返回-1
例子	1、sta 模式获取当前的主要发包速率 <code>iwpriv wlan0 common get_maxrate</code> 2、ap 模式获取与连接的 sta (00:11:22:33:44:55) 当前的主要发包速率 <code>Iwpriv wlan0 common get_maxrate,00:11:22:33:44:55</code>

1.47 获取出厂的 efuse

命令	<code>iwpriv wlan0 common getefusefirst</code>
参数说明	无
返回值	Get efuse data is [1,80,5,4,4,6,0,0,dc:29:19:4c:81:e7] 80: 是 dcxo 频偏 5,4,4 : gain1,gain2,gain3 dc:29:19:4c:81:e7:mac 地址
例子	

1.48 获取当前的工作信道

命令	<code>iwpriv wlan0 common get_work_channel</code>
参数说明	无
返回值	
例子	

1.49 设置国家码

命令	<code>iwpriv wlan0 common country_code,[country]</code>
参数说明	country:只支持如下三个参数 CN : 中国, 支持 1~13 信道 US : 美国, 支持 1~11 信道 JP : 日本, 支持 1~14 信道
返回值	
例子	设置国家码为美国 <code>ifconfig wlan0 up</code> <code>iwpriv wlan0 common country_code,US</code>

1.50 获取国家码

命令	<code>iwpriv wlan0 common get_country_code</code>
参数说明	country: 只支持如下三个参数 CN : 中国, 支持 1~13 信道 US : 美国, 支持 1~11 信道 JP : 日本, 支持 1~14 信道
返回值	
例子	设置国家码为美国 <code>ifconfig wlan0 up</code> <code>iwpriv wlan0 common get_country_code</code>

1.51 获取频偏和 dcxo 寄存器

命令	<code>iwpriv wlan0 common get_cali_data</code>
参数说明	无
返回值	
例子	<code>iwpriv wlan0 common get_cali_data</code>

1.52 与 6441 私有数据通信

(1) 设置 probe request 插入 ssid/密码

命令	<code>iwpriv wlan0 common user_vendor_ie,ssid,psk</code>
参数说明	ssid: 可连接的 ap 的 ssid psk: 可连接的 ap 的密码
返回值	如果 WIFI 有工作信道则只在本工作信道发送, 如果无工作信道则在 1~14 信道发送
例子	<code>iwpriv wlan0 common user_vendor_ie,yzh_test,12345678</code> 抓空中包能看到发送的 probe req 携带私有数据, 私有 IE 为 221

(2) 监听 probe request 帧

命令	<code>iwpriv wlan0 common listen_probe_req,channel</code>
参数说明	channel: 需要监听的信道, 如果 sta 已经连接上 ap, 那么监听信道只能在 ap 所在的信道, channel 参数无效

	注意： wlan0 只能是 sta 模式
返回值	
例子	<code>iwpriv wlan0 common listen_probe_req,1</code> 在 1 信道监听接收 probe req 帧

(3) 获取 probe request 帧携带的数据

命令	<code>iwpriv wlan0 common get_user_vendor_ie</code>
参数说明	无
返回值	
例子	<code>iwpriv wlan0 common get_user_vendor_ie</code> 如果有监听到数据： <code>atbm_ioctl_get_vendor_ie : ssid[yzh_test],password[12345678]</code> 如果没有数据： <code>atbm_ioctl_get_vendor_ie : ssid[],password[]</code>

1.53 获取信噪比

指令	<code>iwpriv wlan0 common get_snr</code>
参数	无
功能描述	获取设置的 rts duration 值。
返回值	获取成功打印: <code>lmac get snr_val = %d ,real snr = 10*log10(4096/%d)</code> 失败打印: <code>snr get fail</code> WIFI4 中: 有效数据获取出来的 snr 值需要在应用端进行计算出实际有效的 snr: <code>snr = 10*log10(4096/ snr_val)</code> WIFI6 中该参数结果为 EVM
指令示例	<code>iwpriv wlan0 common get_snr</code>

1.54 获取底噪

指令	<code>iwpriv wlan0 common get_noise_level</code>
参数	无
功能描述	获取底噪
返回值	打印:

	底噪有效打印： <code>exact noise_level_dBm = %d</code> 底噪是无效数据的打印： <code>initg = %d,reckon noise_level_dBm = %d</code> 失败打印: <code>noise get fail</code>
指令示例	<code>iwpriv wlan0 common get_noise_level</code>

1.55 连续发送携带私有数据的 probe resp

指令	<code>iwpriv wlan0 common send_probe_resp,ssid,psk</code>
参数	ssid: ap 要广播的 ssid psk: ap 的密码
功能描述	Ap 模式下使用有效,在 ap 模式下以间隔为 5ms 的时间一直发送携带私有数据的 probe resp 包
返回值	成功返回 0, 失败非 0
指令示例	Hostapd.conf 配置的 ssid 和 psk 为: yzh_test 12345678 <code>iwpriv wlan0 common send_probe_resp,yzh_test,12345678</code>

1.56 停止连续发送携带私有数据的 probe resp

指令	<code>iwpriv wlan0 common stop_probe_resp</code>
参数	无
功能描述	AP 模式下使用, 停止连续发送携带私有数据的 probe resp
返回值	成功返回 0, 失败返回非 0
指令示例	<code>iwpriv wlan0 common stop_probe_resp</code>

1.57 修改 ap 信道

指令	<code>iwpriv wlan0 common change_chan,chan,ht40</code>
参数	chan: 信道范围: [1,14] ht40: 是否是 HT40 模式: 0, 非 HT40 1, HT40 模式
功能描述	AP 模式下使用, 不用重启 hostapd 实现切换信道, 必须使用高拓提供的 hostapd 才

	行， hostapd -v 显示为: hostapd_xx_ ATBM _XX_XXXXXX
返回值	成功返回 0，失败返回非 0
指令示例	切换到 1 信道 HT20 模式 iwpriv wlan0 common change_chan,1,0 切换到 1 信道 HT40 模式 iwpriv wlan0 common change_chan,1,1

1.58 Ap 模式下手动设置 TIM

指令	iwpriv wlan0 common set_tim,ena,val
参数	ena: 0 : 关闭手动模式，开启自动模式 1 : 开启手动模式，关闭自动模式 val: 十六进制方式设置 Bitmap control & Partial Virtual Bitmap 的值，其中低 8bit 为 Bitmap control 控制位，可以设置为 0/1。 AltoBeam wifi Partial Virtual Bitmap 有 3 字节共 24bit 控制位，当连接的 sta 个数少于 8 个时候，可以控制的只有 8bit，当连接的 sta 个数超过 8 个，可以控制的 16bit。
功能描述	AP 模式下使用
返回值	成功返回 0，失败返回非 0
指令示例	将第连接上 ap 的第 1 个 sta 和第 3 个 sta 的 TIM bitMap 置位 iwpriv wlan0 common set_tim,1,0a00 b7-----b0 05 ==> 0000 1010 Bit0 对应的是 aid0，该 bit 在协议上为保留位不使用。 <pre>/tmp/bin # /tmp/bin # iwpriv p2p0 common set_tim,1,0a00 [atbm_log]:val set [atbm_log]:get data : ena:1, val:a p2p0 common: set_tim success</pre>

1.59 获取手动设置的 TIM 数据

指令	iwpriv wlan0 common get_tim
参数	无
功能描述	AP 模式下使用
返回值	成功返回 0，失败返回非 0

指令示例	<pre>/tmp/bin # /tmp/bin # iwpriv p2p0 common set_tim,1,0a00 [atbm_log]:val set [atbm_log]:get data : ena:1, val:a p2p0 common: set_tim success /tmp/bin # /tmp/bin # /tmp/bin # iwpriv p2p0 common get_tim p2p0 common: beacon tim :customer control tim,Bitmap control:0x00 , Partial Virtual Bitmap:0xa /tmp/bin #</pre>
------	---

1.60 获取 rts duration

指令	iwpriv wlan0 common get_rts_duration
参数	无
功能描述	获取 rts duration
返回值	
指令示例	iwpriv wlan0 common get_rts_duration



CONTACT INFORMATION

AltoBeam (China) Inc.

Address: B808, Tsinghua Tongfang Hi-Tech Plaza, Haidian, Beijing, China 100083

Tel: (8610) 6270 1811

Fax: (8610) 6270 1830

Website: www.altobeam.com

Email: support@altobeam.com

DISCLAIMER

Information in this document is provided in connection with AltoBeam products. No license, express or implied, by estoppels or otherwise, to any intellectual property rights is granted by this document. Except as provided in AltoBeam's terms and conditions of sale for such products, AltoBeam assumes no liability whatsoever, and AltoBeam disclaims any express or implied warranty, relating to sale and/or use of AltoBeam products including liability or warranties relating to fitness for a particular purpose, merchantability, or infringement of any patent, copyright or other intellectual property right. AltoBeam may make changes to specifications and product descriptions at any time, without notice.

Designers must not rely on the absence or characteristics of any features or instructions marked "reserved" or "undefined." AltoBeam reserves these for future definition and shall have no responsibility whatsoever for conflicts or incompatibilities arising from future changes to them.

Unauthorized use of information contained herein, disclosure or distribution to any third party without written permission of AltoBeam is prohibited.

AltoBeam™ is the trademark of AltoBeam. All other trademarks and product names are properties of their respective owners.



Copyright © 2007~2020 AltoBeam, all rights reserved